



THE JOURNAL OF INFORMATION WARFARE

Software-Defined Networking Traffic Engineering Process for Operational Technology Networks

Author(s): WJ Hutton III, AD McKinnon and MD Hadley

Source: *Journal of Information Warfare*, Winter 2019, Vol. 18, No. 4, Preparing for a Future of Critical Infrastructure Under Autonomic Control: SPECIAL EDITION (Winter 2019), pp. 167-181

Published by: Peregrine Technical Solutions

Stable URL: <https://www.jstor.org/stable/10.2307/26894699>

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



JSTOR

Peregrine Technical Solutions is collaborating with JSTOR to digitize, preserve and extend access to *Journal of Information Warfare*

Software-Defined Networking Traffic Engineering Process for Operational Technology Networks

WJ Hutton III, AD McKinnon, MD Hadley

*Pacific Northwest National Laboratory
Richland, Washington, United States*

E-mail: william.hutton@pnnl.gov; david.mckinnon@pnnl.gov; mark.hadley@pnnl.gov

Abstract: *Network designs are often functional with little consideration for security. Growth and maintenance of critical infrastructure, mission-focused networks can be ad hoc and can preclude large-scale technology replacements, reconfigurations, or even patching. These restrictions are contributing factors to increased vulnerabilities. Over 90% of all attacks begin with phishing and include lateral movement (Frincke 2019). Adopting a Software-Defined Networking (SDN) approach is an effective mitigation strategy for many existing vulnerabilities. Software-Defined Networks use a ‘deny-by-default’ approach to networking that restricts an adversary’s freedom of movement without impacting an organisation’s mission. SDN implementations require a process that identifies an organisation’s mission-specific network communications. A well-defined traffic engineering process ensures that only known devices communicate with each other on specific ports.*

Keywords: *Software Defined Networking (SDN), Situational Awareness, Network Security, Cybersecurity*

Introduction

Unlike conventional warfare in natural domains of land, sea, air, and space, topology in the cyber domain is dynamic and is easily manipulated by humans. An adversary can create connections between two points and can move through a network in unexpected ways that bypass carefully designed security controls. As such, the need to gain and control key terrain (high ground)—for both defensive and offensive purposes—goes beyond the geographic and physical to include the logical and cyber personas which define and influence the domain’s topology.

Many Operational Technology (OT) networks—networks that are designed to support physical devices and processes—often lack even the most basic documentation, such as network diagrams. The Information Technology (IT) department may not know what field devices are deployed on these OT networks, what protocols these devices are using, or what ports they are using to communicate to other devices. Some networks that were believed to be air gapped may even be communicating with devices on the Internet. This paper outlines the approach to accomplishing three primary Software-Defined Networking (SDN) traffic engineering objectives:

- Automatic generation of a network ground truth from communications occurring in an OT network,
- Presenting that ground truth in a simple-to-understand way to non-technical stakeholders, and
- Rapid creation of SDN flow rules to permit authorised communications in a deny-by-default paradigm.

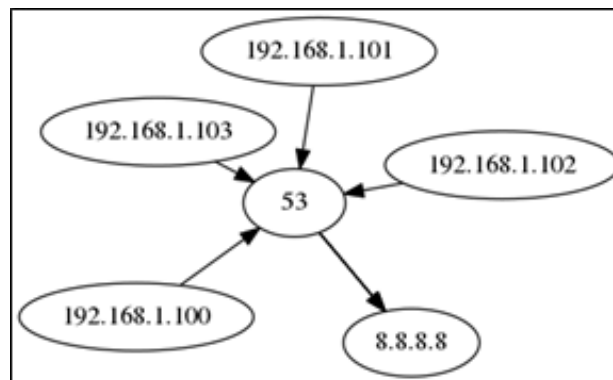


Figure 1: A simple network graph

Graph theory is useful for establishing a technical *lingua franca* and will be useful for analysing topologies. **Figure 1** shows a network graph, which is a set of nodes connected by one or more edges. A node (shown as an ellipse) is a fundamental unit and can represent any object. An edge (line) connects two nodes and represents a relationship between those objects. Edges can be unidirectional, bidirectional, or could have no direction. The direction of an edge is indicated by an arrowhead at one or both ends of the line. The shape of a graph is often referred to as the graph's topology.

The network graph in **Figure 1** shows how different nodes in the same graph can represent multiple things, for example, IPv4 addresses (sources and destinations) and the service port (an integer). The relationship between these three things can be understood by the unidirectional edges that connect them. Port 53 is a well-known service port for a Domain Name System (DNS). Source addresses can be identified by the multiple incoming edges pointing to the 53 node. The 53 node also has a single outgoing edge pointing to the 8.8.8.8 node, which is the IPv4 address for Google's DNS server. This graph shows four hosts, each with a unique IPv4 address, connected via port 53 to Google's well-known DNS server at IPv4 address 8.8.8.8.

Related Work

OT networks are purpose-built, typically air gapped, and tend to exhibit slow, organic growth and change over time. OT networks may include legacy devices and vendor-specific protocols that are not widely used and are no longer supported. Often OT networks are designed and deployed by third-party consultants. Engineering an OT network prior to SDN deployment is critical to a successful deployment.

The best source of ground truth data regarding what devices are on the network and how they are communicating with each other is live network packet capture files. It is also important to exercise the network to ensure that each device communicates so that both normal and abnormal behaviour can be observed. For example, if a hot standby server exists, but no communication between field devices and the standby server are observed, the corresponding flow rules cannot be created, and resiliency will be negatively impacted.

Imperfect data can result in gaps of knowledge about the OT when implementing flow rules. This can be mitigated by tagging network traffic entering the flow controller with Virtual Local Area Network (VLAN) IDs. For example, traffic that matches an existing flow is assigned a VLAN ID of 124 (T). Traffic that is false for an existing rule is assigned a VLAN ID of 106 (F). Network traffic that does not correspond to any current flow is known as a miss and is assigned a VLAN ID of 115 (M). Additional flow rules are added to address the VLAN ID 115 traffic. The goal is to not have any flow rule misses.

There are no tools specific to the traffic engineering process for an OT-SDN network. An OT-SDN engineering tool should present a graphic representation of actual network traffic. This makes planning the network and flow rules more efficient. The tool must be able to ingest packet capture files, which are ground truth of the network conversations between field devices.

Network visualisation

Several existing tools, such as Cacti (Cacti 2019), EtherApe (EtherApe 2019), NtoPNG (Ntop 2018), and PcapViz (Mateuszk87 2018) were evaluated for their suitability to meet these network visualisation needs based on the following criteria:

- The ability to ingest network packet capture files,
- A graphical representation on the network topology,
- An Application Programming Interface (API) or source code that could be extended to integrate with SDN vendors.

The Cacti tool only visualises streaming network traffic and cannot import packet capture files. EtherApe has not been updated since 2018 and was not successfully installed due to multiple dependency issues. NtoPNG is also a streaming network visualisation tool and did not have the ability to import *.pcap files. PcapViz specifies that it is “not suited for big datasets”; it crashed after trying to import a 271-megabyte *.pcap file for 15 minutes. Some OT devices communicate infrequently, requiring lengthy network packet captures that can result in *.pcap files much larger than 271 megabytes.

Firewalls

SDN functionality could also be implemented via network-based firewalls. Default firewall policy can be implemented in one of two ways: a default policy of ‘accept’ with blacklists (individual rules that block specific types of traffic) or a default policy of ‘deny’ with whitelists (individual rules that allow specific types of traffic). Both approaches require an administrator to explicitly define every rule and increase complexity, making the network brittle or fragile.

Each new type of conversation between existing network devices and every new network device will require new firewall rules, greatly increasing maintenance time for firewall administrators. Thus, firewall rules used to control the flow of traffic between subnets or networks often use Classless Inter-Domain Routing (CIDR) notation, which allows network traffic at the subnet level rather than the per-device level. This results in a much larger attack surface (see the ‘SDN and the Deterministic Quality of OT Network Traffic’ section below) and allows an attacker to pivot from one network device to another, which is not possible when SDN is deployed with a ‘deny-by-default’ approach.

Many OT devices are resource constrained and are unable to implement host-based firewalls or perform deep packet inspection. Cyber-physical systems often have strict timing requirements that also limit additional processing. Managing host-based firewalls for each field device in an OT network would also increase the complexity and time requirements for installation and maintenance of field devices when compared to centralising these tasks in an SDN network controller.

Like bastion hosts, network-based firewalls are also subject to remote compromise and reconfiguration. Compromise and reconfiguration by an attacker with physical access to most traditional network devices is trivial. However, even physical access may not be enough to compromise an SDN network that has been configured and deployed with security in mind.

Existing rules may only be modified via the controller, and new rules may only be added using the controller. Neither is possible once the controller has been removed from the SDN network. Flow rules lock in known good network behaviour. Flow rules to explicitly forbid remote connections are also prudent. SDN switches may be configured to drop network packets that do not correspond to any existing flow rule (that is, a miss). Each SDN switch within the network is adopted by one and only one SDN controller. Once adopted, the SDN switch will no longer use the OpenFlow discovery protocol, removing a potential source of network reconnaissance for the attacker. Any attacker wishing to spoof the original SDN controller to compromise SDN switches would need to mimic the properties of the original controller, including the exact certificates used for authentication and Transport Layer Security (TLS) encryption.

SDN and the Deterministic Quality of OT Network Traffic

Large IT networks (such as Tier 1 Internet service providers) use SDN technology because they cannot deploy and configure classical networking hardware quickly enough to meet the dynamic topology needs of their customers. In contrast, most OT network traffic is predictable, including communications by the same sources (field devices) transmitting network packets of the same size at regular intervals to the same destination (a server). Furthermore, most OT networks tend to be small both in the number of networked devices and their connectedness. Network devices are not often added or removed, and the configuration of these devices tends to be static. These properties allow for the use of SDN in a completely different way to provide deny-by-default security.

Complexity

The finite automata computational model is useful to understand the attack surface in a network. Based on observed network traffic (a given *.pcap file), a graph that describes the communication relationship between all devices on the network is easily generated. Normally, only one graph

would be used to visualise the traffic of a network; but to increase readability here, the SSH, HTTPS, and DNS network traffic are split into subgraphs (see **Figures 2a, 2b, and 2c**, below). To grasp the reduction in attack surface that SDN provides for OT networks, imagine that the network communications specification is a Deterministic Finite Automaton (DFA) machine. By adding unique symbols to each edge in the graphs, an alphabet was created for the DFA. In the case of the network shown in **Figures 2a, 2b, and 2c**, $\Sigma\text{SDN} = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, r, s, t\}$. Every word in the language accepted by this DFA is a digram (a sequence of two letters). In total, there are $19C2 = 171$ possible digrams in this language, but only 17 of those diagrams are accepted by the DFA. This small set of digrams is expected to be generated with a high probability in normal operations. Likewise, the remaining 90% of digrams should be generated with a probability that approaches zero for normal operations. For example, in **Figure 2c**, lt, mt, nt, ot, pt, qt, rt, and st are the only valid digrams that represent the eight source addresses allowed to use DNS, and they may only resolve DNS to the destination address represented by node t, which is Google's DNS server.

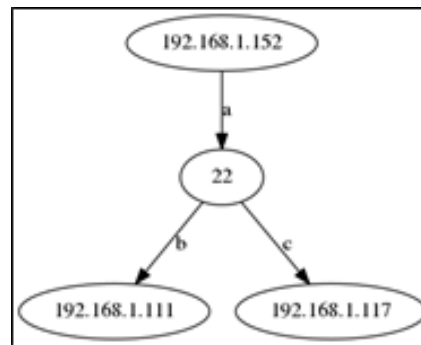


Figure 2a: DFA subgraph of network SSH traffic

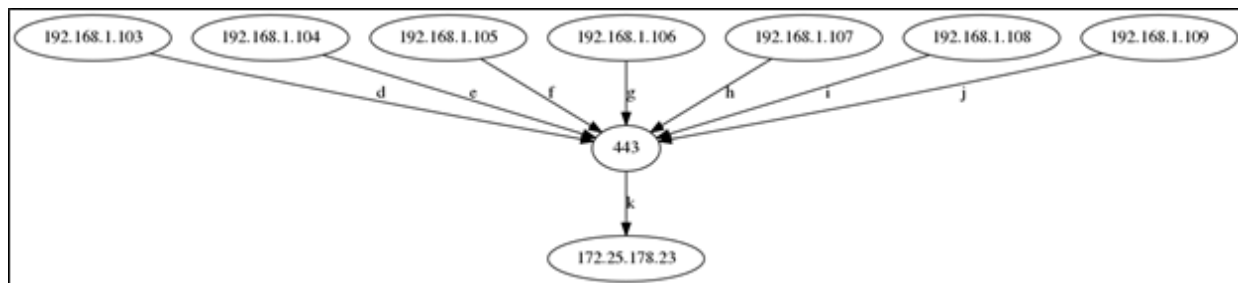


Figure 2b: DFA subgraph of network HTTPS traffic

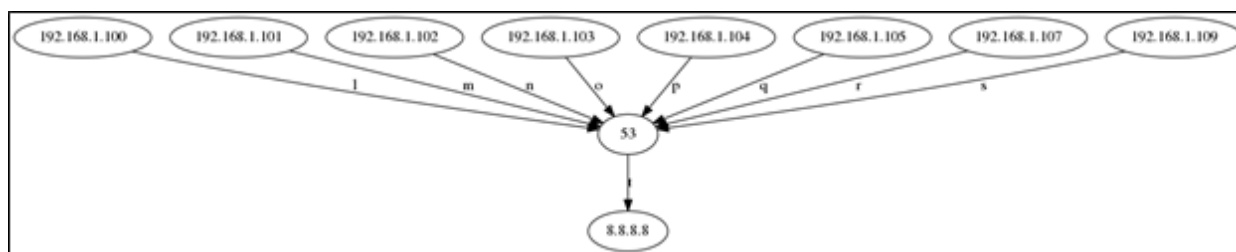


Figure 2c: DFA subgraph of network DNS traffic

Contrast this with a typical network-based firewall. Recall the cardinality of the SDN alphabet is $|\Sigma_{\text{SDN}}| = 19$. A typical firewall rule allowing a class C subnet access to a single network resource would result in $|\Sigma_{\text{FW}}| = 254 + 1$ or 255 symbols, even if the class C network is not fully utilised. A common physical attack begins by connecting an unauthorised device to the network. For the example SDN, there are still only 17 authorised words ($|L(\text{SDN})| = 17$). A direct connection of $|L(\text{FW})| = 254$; but with just two pivots, the cardinality of the resulting four-gram words in $|L(\text{FW})|$ explodes to 32,131 because the attacker has $254C2$ choices on how to pivot between a compromised host and a target host. Because pivoting is not authorised in SDN, the cardinality of possible connections remains unchanged from the original specification of a manageable 17.

The stack abstract data type is also useful as a metaphor to understand the problem of observability for situational awareness (detection and validation). Recall that a stack is a first-in, last-out collection of elements where only the top (or last) element is visible. If the DFA's language is enforced with a firewall, the only elements available are source addresses, destination addresses, and destination ports. A firewall rule is a simple Boolean evaluation of a network packet based on these elements.

A typical firewall rule allows any device on the network to resolve DNS queries, but that behaviour could be restricted to just seven network devices using stricter firewall rules. Following the principle of least privilege (Confluence 2018), this best practice can mitigate some types of protocol misuse, like Night Dragon (McAfee Foundstone Professional Services and McAfee Labs 2011), which leveraged DNS as a command and control mechanism. SDN provides finer control over defining relationships. SDN flows can use additional properties, such as frequency and size of network packets, to further define a communication relationship.

Deterministic network traffic simplifies anomaly detection, but SDN can help prevent it by shortening the observation and orientation steps of the Observe, Orient, Decide, and Act (OODA) loop and automating the decision and action steps. All traffic conforms to the specification implemented by the network flows, or it is dropped. The worst-case scenario is that an adversary can spoof a device and communicate with it as expected.

Use case

SDN technology is an ideal solution to secure Industrial Control Systems (ICS) and Supervisory Control And Data Acquisition (SCADA) systems due to the deterministic behaviour of the networks used to monitor and control the physical process. The hardware and software components of

these ICS and SCADA systems are designed to perform the same function in the same manner for decades. A typical U.S. Department of Defense installation contains many ICS or SCADA systems where SDN can be deployed. Consider the following sample use cases:

- Microgrids,
- Base electric distribution,
- Backup power generation,
- Energy storage,
- Renewable energy integration,
- Potable water,
- Wastewater,
- Building automation (air and lighting controls),
- Fire suppression,
- Oil and natural gas,
- Physical security (cameras, proximity readers, various sensors).

SDN is also applicable to command and control environments used to support other U.S. Department of Defense missions. Similar to ICS and SCADA systems, command and control system hardware and software components exhibit deterministic behaviour. Some example systems where SDN is applicable include satellite base stations, electric distribution on ships, and field deployable microgrids.

OT Traffic Analysis

Securing networks within critical infrastructure is a unique cybersecurity challenge. These networks usually consist of a combination of analogue and digital components and may span multiple decades of differing technologies. Initial design goals were often focused on functionality with little to no consideration for security. Maintaining critical infrastructure capabilities can preclude large-scale technology replacements, reconfigurations, or even patching. Each of these restrictions is a contributing factor to increased vulnerability and adversarial impact. Over 90% of all cybersecurity attacks begin with phishing and include some type of lateral movement (Frincke 2019). SDN has proven to be an effective mitigation by taking a deny-by-default approach in deterministic network environments as one possible means of addressing unauthorised lateral movement.

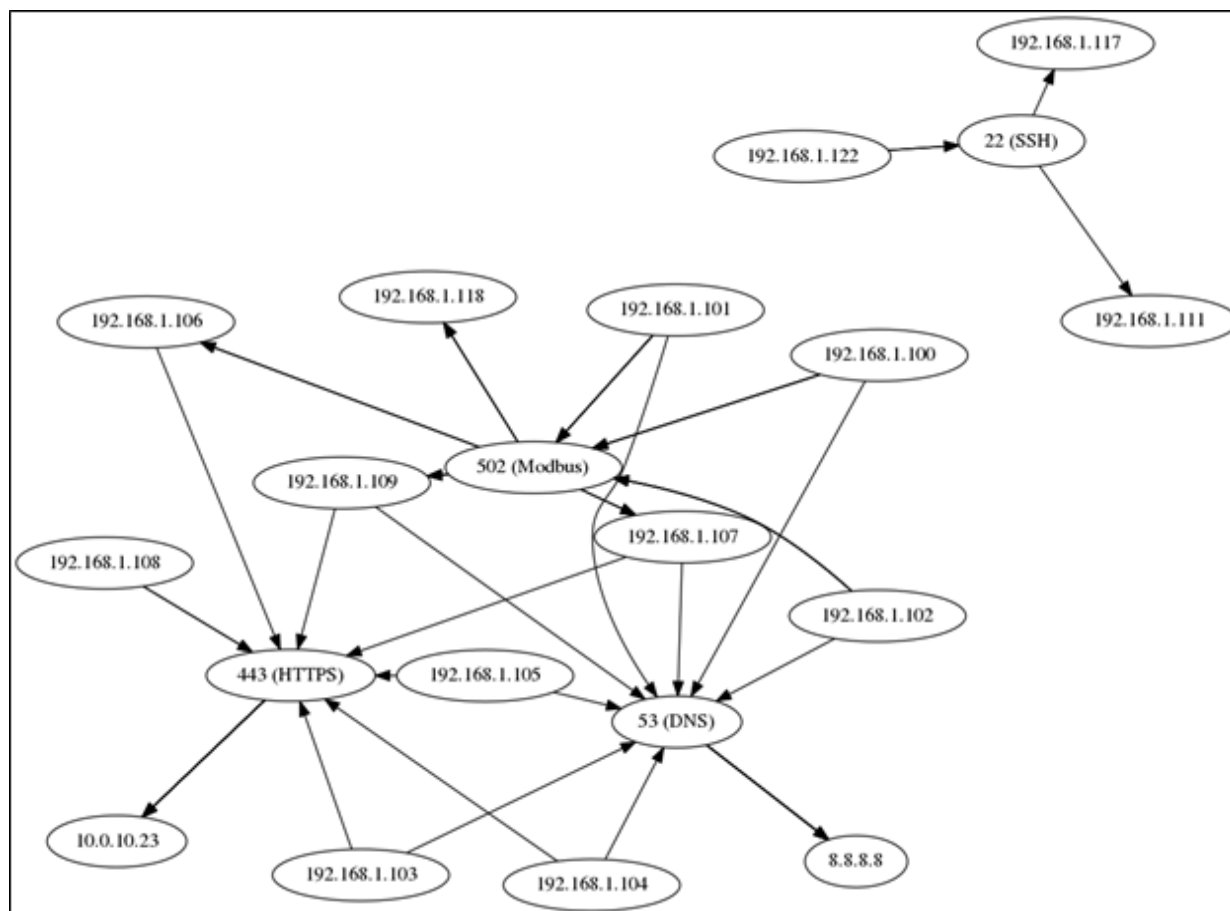


Figure 3: Typical OT network traffic graph

Network example

For illustration purposes, a sample OT network that communicates using Modbus, Secure Shell, Secure HTTP, and DNS is shown in **Figure 3**, above. The first question to consider in traffic analysis is, “What network traffic do you expect to find on the network?” This initial hypothesis will be validated with network packet capture, network diagrams, and meetings with site network engineers (see the ‘SDN approach to OT traffic engineering section’, below).

SDN approach to OT traffic engineering

There are different approaches to gaining situational awareness and locking down an OT network with SDN. This approach has two design goals. First, minimise manual processes—avoid writing things down or manually editing lengthy spreadsheets and log files. Second, a pictorial representation of the network is crucial to get input from non-network engineers. The following is a summary of the approach to this problem:

- Capture raw network traffic (*.pcap files) with tcpdump.
- Generate a list of features (such as source addresses, source ports, destination addresses, and destination ports) from the *.pcap files.

- Determine which port for a given network connection is the service port (see the ‘determine service port’ section).
- Generate a graph (see **Figures 2 and 3**).
- Source addresses are nodes with an outgoing arrow to a service port node.
- Service port nodes have an outgoing arrow to a destination address.

These four steps are discussed in detail below.

Step 1: Capture raw network traffic. The first step of capturing raw network traffic is likely familiar to many. A SPAN or mirror port should be configured and then used to ensure all OT network traffic is captured. The capture process should be allowed to run for a relatively long period because some OT devices may be configured to communicate infrequently. Using the tcpdump utility, capturing network packets can be started by issuing the following simple command:

```
tcpdump -i [interface] -w [filename]
```

This command could be modified to stop packet capture after a number of packets (say 1,000,000 with the -c 1000000 option), after a certain amount of time (like 1 hour with the -G 3600 option) or when the *.pcap file reaches a certain size (perhaps 500 megabytes with the -C 524.288 option). If multiple *.pcap files are captured, initial analysis can be conducted on each file or the files can be merged into a single file for a complete picture of the network. Merging files can be accomplished with the ‘mergcap’ command:

```
mergcap [file 1] [file ..] [file n] -w [output file].pcap
```

Note the mergcap and tshark commands are both part of the command-line tools installed with a Wireshark download (Wireshark 2019).

Step 2: Generate features. Although the binary format of Wireshark output files is well understood, software tools that parse text files are easier to understand and maintain. The first method explored to create a text file was to extract individual features from the binary packet capture using the tshark command.

The tshark command can extract specific fields such as the source addresses from the *.pcap file:

```
tshark -r [file.pcap] -T fields -e eth.src -e ip.src  
-E separator=, -E quote=d > [output1.csv]
```

Likewise, the destination addresses can also be extracted:

```
tshark -r [file.pcap] -T fields -e eth.dst -e ip.dst  
-E separator=, -E quote=d > [output2.csv]
```

Then both files are merged by using the cat command:

```
cat [output1.csv] [output2.csv] > [output.csv]
```

Finally, the sort command can be used to sort the list and to discard duplicate addresses.

```
cat [output.csv] | sort -u > [uniqueOutput.csv]
```

There is a loss of some information that may be valuable, as conversations between two devices where only one packet was exchanged are given the same weight as a conversation that may occur 720 times a day.

Information from the resultant file can be used to verify assets by physically walking down the OT network looking for each host mentioned in the file. Additionally, this file's data can be used to generate a graph that visualises the network traffic and associated flows that will need to be authorised by the SDN controller.

For Python, the tshark command can also be used to convert the binary *.pcap file to JavaScript Object Notation (JSON), which is a structured text file format. There is a very large increase in file size, approximately 20 times larger (a 6 MB *.pcap file results in a 115 MB *.json file). Thankfully, Python is very efficient at parsing text files and it takes less than 3 seconds to parse the 115-mega-byte *.json file on an Apple MacBook Pro, without any heuristics for speed:

```
tshark -T json -r [file.pcap] > [file.json]
```

Using Python to parse JSON and generate a Graphviz (*.gv) file allows some useful analytics and draws a better graph for situational awareness. Additional improvements are enumerated in 'Future Work section' below.

Use of a scripting language such as Python also allows for the automation of the various commands (for example, tcpdump, tshark, and Graphviz) as well as interfacing with vendor APIs to automatically create SDN flow controller rules and present them to a human-in-the-loop model for verification (see the 'Future Work' section below).

Step 3: Determine the service port. Determining the service port is a problem compounded by multiple standards that have existed over time. Prior to 1998, the service port selection standard was governed by the University of Southern California's Information Sciences Institute; the Internet Corporation for Assigned Names and Numbers (or ICANN) governed from 1998–2001; and finally, the Internet Assigned Numbers Authority has been setting the standards since 2001. The current service port standards are

- Ports 0 – 1,023: system or well-known ports,
- Ports 1,024 – 49,151: user or registered ports,
- Ports 49,152 – 65,535: dynamic or private ports.

It is the 'or' in the service port standards listed above that is problematic. Ephemeral ports are selected using different schemes depending on an operating system vendor's implementation:

- Linux: 32,768 – 61,000 -- local, random;

- Microsoft Windows XP: 1,024 – 5,000 -- global, sequential;
- Microsoft Windows 7: 49,152 – 65,535;
- Apple OS X 10.9: 49,152 – 65,535 -- global, Transmission Control Protocol (TCP) sequential, User Datagram Protocol (UDP) random.

Consider the source port selection scheme used by Windows XP, which can still be found widely deployed in OT networks. Port 3389 is a registered port for Microsoft's Remote Desktop Protocol and is a valid user port. If port 3389 is already in use, 3390 would be tried next because of XP's sequential assignment strategy. This makes determining the service port in a conversation difficult. A naive algorithm is shown in Equation 1, below.

Equation 1: Naive Service Port Algorithm

```
def ChooseServicePort(port1, port2):
    if port1 < 1024 and port2 >= 1024:
        servicePort = port1
    if port2 < 1024 and port1 >= 1024:
        servicePort = port2
    if port 1 >= 1024 and port2 >= 1024:
        if port1 < port2:
            servicePort = port1
        if port2 < port1:
            servicePort = port2
    return servicePort
```

A list of known service ports in use on the network can be added to the Python script. Then if port1 or port2 is in this list, the algorithm in Equation 1 can be modified to return the known service port by making the lines the first conditionals as in Equation 2, below.

Equation 2: Naive Service Port Algorithm

```
if port1 in SERVICE_PORTS:
    return port1
if port2 in SERVICE_PORTS:
    return port2
```

Combining Equations 1 and 2 still results in a naive approach to solving the problem of determining the service port. The result is a purpose-built algorithm that is usually correct and sufficient. When the algorithm fails, the resulting graph can appear strange, with a large number of sequential ports connected to the same destination address—a condition that a traffic engineer can easily identify. The engineer can then quickly and correctly identify the service port using a Wireshark filter, after which this port can be added to the list of known ports. This eliminates incorrectly reported service ports when the service port is higher than the source port (where source port = 1028, destination port = 3389).

Using the TCP three-way handshake, a packet with just the SYN flag set and the destination port of that packet could be used as the service port for the conversation. Unfortunately, only about 30%

of the OT network traffic observed was TCP based, so this approach is of limited value.

Step 4: Generate a graph. Generating a graph to visually depict OT network traffic is easy. The Graphviz command syntax is straightforward and the complete command line to convert the notation to a PNG file is simple:

```
neato -T png -o [file].png [file].gv
```

A key constraint to the graph generation process is the number of nodes in the graph. The current process does not parallelise well, and the time required to generate the same graph did not differ significantly using a modest virtual machine or the 76th largest supercomputer (Top500 2018). A well-behaved OT network should not approach this limit of approximately 300 nodes, but most OT networks are not well-behaved. As can be seen in **Figure 3**, above, network conversations may consist of multiple disjoint graphs, with some conversations occurring at level 2 of the Open System Interconnection model using the Address Resolution Protocol, some conversations using IPv4 addresses, and others using IPv6 addresses (not shown). There may also be a mix of unicast, multicast, and broadcast traffic, along with multiple private address ranges and some public addresses.

An OT networking best practice is to air gap (or isolate) the OT networks. Unfortunately, network engineers may not know that their OT network has been erroneously or maliciously configured to talk to a remote host located on the Internet at large. Furthermore, constant vigilance is required to ensure only authorised communication is occurring. For example, is the firewall only allowing DNS traffic that supports the authorised querying of device names, or is an adversary exfiltrating data using a covert channel masquerading as DNS traffic?

SDN Flow Controller Rules Creator

The list of rules below was automatically generated from **test.pcap**.

Click the **Create** button if you approve of the rule or the **X** button if you disapprove (and the rule will not be created).

Easy mode

☒ On ☐ Off

Source Address	Destination Address	Destination Port	
192.168.1.152	192.168.1.111	SSH	Create X
192.168.1.152	192.168.1.117	SSH	Create X
192.168.1.100	dns.google.com	DNS	Create X
192.168.1.101	dns.google.com	DNS	Create X
192.168.1.102	dns.google.com	DNS	Create X
192.168.1.103	dns.google.com	DNS	Create X
192.168.1.104	dns.google.com	DNS	Create X
192.168.1.105	dns.google.com	DNS	Create X
192.168.1.107	dns.google.com	DNS	Create X
192.168.1.109	dns.google.com	DNS	Create X
192.168.1.103	172.25.178.23	HTTPS	Create X
192.168.1.104	172.25.178.23	HTTPS	Create X
192.168.1.105	172.25.178.23	HTTPS	Create X
192.168.1.106	172.25.178.23	HTTPS	Create X
192.168.1.107	172.25.178.23	HTTPS	Create X
192.168.1.108	172.25.178.23	HTTPS	Create X
192.168.1.109	172.25.178.23	HTTPS	Create X

Figure 4: Proposed GUI interface for automated flow controller rules creation

Conclusion

OT networks have different vulnerabilities and challenges that must be mitigated when compared with traditional IT networks. The deterministic communication patterns and relatively small, unchanging number of field devices in OT networks, combined with their limited external connectivity, are ideal for a deny-by-default SDN deployment. When viewed as a DFA, the language an SDN accepts is orders of magnitude smaller than that of a typical IT firewall, which greatly reduces the SDN's attack surface and can prevent adversarial pivoting. One challenge to creating the flow rules necessary to deploy an SDN is visualising and understanding what network traffic should be allowed on the OT network. The approach described in this paper can be leveraged by anyone who would like insight into the conversations happening on their networks. This information can be leveraged to create flow rules for any SDN controller, or automated using a vendor API to reduce technical barriers to deploying SDN and securing critical infrastructure and cyber-physical systems.

Future Work

The authors plan to automate the creation of flow rules from raw network packet capture files with at least one vendor's SDN controller API. A human traffic engineer will be kept in the loop to verify and authorise each SDN flow rule. To reduce cognitive load, raw IP addresses can be replaced with fully-qualified domain names, and well-known service port numbers can be replaced with their service name. This type of simplification may make some security professionals uncomfortable, so users may toggle these options on and off with an 'easy' mode (see **Figure 4**, above).

Some improvements considered for the Python scripts and network graph visualisation described previously include

- Counting the number of conversations between addresses and using thicker lines to draw edges between these nodes,
- Filtering addresses by count (graphing only addresses where one packet was exchanged—often an indication of malicious activity),
- Colouring nodes based on a predefined specification (for example, yellow for unknown local devices and red for unknown external devices),
- Predefining a service port list for more accurate graphs,
- Examining TCP flags to accurately detect service ports,
- Using vendor MAC lookup for asset identification.

Acknowledgements

The work described in this paper was conducted under the sponsorship of the U.S. Department of Energy by cybersecurity researchers and engineers at Pacific Northwest National Laboratory. The authors extend their gratitude to the Cybersecurity for Energy Delivery Systems Division within the Cybersecurity, Energy Security, and Emergency Response Office and to the Department of Defense stakeholders, electric utilities, vendors, and engineering firms who have partnered on SDN projects.

References

Cacti 2019, 'Cacti® - The complete RRDTool-based graphing solution', viewed 25 June 2019, <<https://www.cacti.net/>>.

Confluence 2018, 'Top 10 secure coding practices', viewed 25 June 2019, <<https://wiki.sei.cmu.edu/confluence/display/seccode/Top%2B10%2BSecure%2BCoding%2BPractices>>.

EtherApe 2019, 'EtherApe, a graphical network monitor', viewed 25 June 2019, <<https://etherape.sourceforge.io/>>.

Frincke, D 2019, *Keynote address, Northwest Cybersecurity Symposium*, Pacific Northwest National Laboratory, 8-10 April, Richland, WA, US.

Mateuszk87 2018, 'PcapViz', *Github*, viewed 25 June 2019, <<https://github.com/mateuszk87/PcapViz>>.

McAfee Foundstone Professional Services and McAfee Labs 2011, *Global energy cyberattacks: "Night Dragon"*, viewed 25 June 2019, <https://securingtomorrow.mcafee.com/wp-content/uploads/2011/02/McAfee_NightDragon_wp_draft_to_customersv1-1.pdf>.

Ntop 2018, 'ntopng: High-speed web-based traffic analysis and flow collection', viewed 25 June 2019, <<https://www.ntop.org/products/traffic-analysis/ntop/>>.

Top500 2018, 'Top500 list – November 2018', viewed 27 June 2019, <<https://www.top500.org/list/2018/11/?page=1>>.

Wireshark 2019, 'Download Wireshark', viewed 25 June 2019, <<https://www.wireshark.org/#download>>.